

Fundamentos y Arquitectura de Aplicaciones Back-End

Back-End Development

Carlos Rojas-Sánchez

Universidad del Mar

Contenido

1. HTTP y modelo cliente-servidor
2. Arquitectura de software en el back-end
3. Estructura y gestión del código fuente

HTTP y modelo cliente-servidor

El modelo cliente-servidor

Concepto fundamental

Es un modelo de arquitectura distribuida donde las tareas se reparten entre los proveedores de recursos o servicios (**servidores**) y los demandantes (**clientes**).

- **Cliente**: Inicia la comunicación enviando una solicitud (*request*).
- **Servidor**: Escucha en un puerto, procesa la solicitud y devuelve una respuesta (*response*).
- **Desacoplamiento**: El cliente no necesita conocer la lógica interna del servidor ni cómo se almacenan los datos.

Protocolo HTTP: Fundamentos

Hypertext Transfer Protocol
Protocolo de la capa de aplicación (modelo TCP/IP) sin estado (*stateless*) que rige el intercambio de mensajes en la Web.

- **Arquitectura Stateless**: Cada petición se trata de manera independiente; el servidor no guarda memoria implícita del cliente entre peticiones.
- **Mecanismos de estado**: Para mantener sesión se recurre a *cookies*, tokens (JWT) o sesiones del lado del servidor.
- **Transporte**: Opera habitualmente sobre TCP (puertos 80 para HTTP y 443 para HTTPS).

Estructura de una petición HTTP (Request)

Una petición enviada por el cliente consta de tres partes principales:

1. **Línea de solicitud**: Incluye el método HTTP, la URI del recurso y la versión del protocolo.
2. **Encabezados (Headers)**: Metadatos como `Host`, `User-Agent`, `Accept`, `Authorization`, etc.
3. **Cuerpo (Body)**: Datos opcionales enviados al servidor (JSON, XML, form-data), común en métodos **POST** o **PUT**.

Estructura de una respuesta HTTP (Response)

La respuesta devuelta por el servidor contiene:

1. **Línea de estado**: Versión de HTTP, código de estado numérico y mensaje asociado.
2. **Encabezados (Headers)**: Información del servidor, tipo de contenido (`Content-Type`), tamaño (`Content-Length`), directivas de caché, etc.
3. **Cuerpo (Body)**: La carga útil o recurso solicitado (documento HTML, objeto JSON, imagen, etc.).

Métodos HTTP principales

Define la acción que se desea realizar sobre el recurso:

- **GET**: Recuperar representación de un recurso (seguro e idempotente).
- **POST**: Enviar datos para crear un nuevo recurso en el servidor.
- **PUT**: Reemplazar completamente un recurso existente o crearlo si no existe.
- **PATCH**: Aplicar modificaciones parciales a un recurso.
- **DELETE**: Eliminar el recurso especificado.

Códigos de estado HTTP

Indican el resultado del procesamiento de la petición en el servidor:

- **1xx (Informativos)**: Petición recibida, continuando el proceso.
- **2xx (Éxito)**: Acción recibida, entendida y aceptada (ej. `200 OK`, `201 Created`).
- **3xx (Redirección)**: Se requieren acciones adicionales (ej. `301 Moved Permanently`).
- **4xx (Error del Cliente)**: La solicitud contiene sintaxis errónea o no se puede procesar (ej. `400 Bad Request`, `404 Not Found`).
- **5xx (Error del Servidor)**: El servidor falló al procesar una solicitud válida (ej. `500 Internal Server Error`).

Encabezados HTTP (Headers) esenciales

Permiten al cliente y al servidor enviar información adicional con la petición o respuesta:

- **Content-Type**: Especifica el tipo MIME del cuerpo (ej. `application/json`).
- **Authorization**: Envía credenciales de autenticación (ej. `Bearer <token>`).
- **Access-Control-Allow-Origin**: Cabecera clave para la política CORS.
- **Cache-Control**: Define directivas para el almacenamiento en caché del cliente o intermediarios.

HTTPS y Seguridad en la capa de transporte

HTTP + TLS/SSL

HTTPS cifra las comunicaciones entre cliente y servidor para garantizar confidencialidad e integridad.

- **Cifrado**: Protege los datos ante ataques de escucha (*eavesdropping*).
- **Integridad**: Impide la alteración no detectada de los datos durante el tránsito.
- **Autenticación**: Uso de certificados X.509 firmados por una Autoridad de Certificación (CA).

Mecanismos de persistenia de estado: Cookies vs Tokens

Al ser HTTP un protocolo sin estado, se emplean dos enfoques para gestionar sesiones:

Cookies / Sesión

- El servidor guarda el estado de sesión.
- El cliente almacena una cookie con el ID de sesión.
- Sensible a ataques CSRF si no se configura correctamente.

Tokens (JWT)

- El servidor no guarda estado (*stateless*).
- El cliente almacena el token firmado.
- Se transmite habitualmente en la cabecera `Authorization`.

Flujo de interacción HTTP completo

1. **Resolución DNS**: El cliente traduce la URL a una dirección IP.
2. **Handshake TCP / TLS**: Establecimiento del canal de transporte seguro.
3. **Envío de Request**: El cliente transmite método, URI, cabeceras y cuerpo.
4. **Procesamiento**: El servidor web/API recibe la petición y ejecuta la lógica back-end.
5. **Envío de Response**: El servidor responde con un código de estado, cabeceras y carga útil.
6. **Cierre/Reuso**: Conexión finalizada o mantenida abierta (`Keep-Alive`).

Arquitectura de software en el back-end

Arquitectura en capas: Visión general

Separación de responsabilidades

Organiza la aplicación en niveles horizontales con responsabilidades bien definidas. Cada capa solo interactúa con la capa inmediatamente inferior.

- **Capa de Transporte / Rutas**: Recibe solicitudes externas y las dirige al controlador correspondiente.
- **Capa de Controladores**: Maneja la interacción HTTP, valida entradas y serializa las respuestas.
- **Capa de Servicios**: Contiene las reglas de negocio puras de la aplicación.
- **Capa de Repositorios**: Aísla el acceso a la base de datos o servicios de almacenamiento.

Desglose de capas: Rutas y Controladores

Rutas (Routing)

- Mapean endpoints HTTP (**GET**, **POST**) a handlers específicos.
- Aplican middlewares de transporte (autenticación, parseo de cuerpo).
- *No contienen lógica de negocio ni acceso a datos.*

Controladores (Controllers)

- Extraen parámetros, *headers* y cuerpo del *request*.
- Invocan a los servicios correspondientes.
- Devuelven códigos de estado HTTP adecuados (`200`, `400`, `404`, `500`).

Desglose de capas: Servicios y Repositorios

Servicios (Services)

- Núcleo del dominio y reglas de negocio.
- Calculan, procesan y coordinan orquestaciones complejas.
- Independientes del protocolo de transporte (pueden invocarse desde HTTP, CLI o tareas programadas).

Repositorios (Repositories)

- Encapsulan la persistencia de datos (SQL, NoSQL, ORM).
- Exponen operaciones tipo CRUD sobre colecciones u entidades.
- Permiten cambiar el motor de base de datos sin alterar la lógica de negocio.

Inversión de Control (IoC) e Inyección de Dependencias (DI)

Principio de Inversión de Dependencias (D de SOLID)

Los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones.

- **Inversión de Control (IoC)**: Delegar la creación y gestión del ciclo de vida de los objetos a un contenedor o marco de trabajo.
- **Inyección de Dependencias (DI)**: Patron donde una clase recibe sus dependencias desde el exterior en lugar de instanciarlas internamente.
- **Beneficios**:
  - Facilita enormemente las pruebas unitarias mediante el uso de *mocks* o *stubs*.
  - Reduce el acoplamiento rígido entre componentes.

Configuración basada en entornos y variables

El principio de los 12 Factores (Factor III: Configuración)
La configuración de una aplicación varía según el entorno (Desarrollo, Pruebas, Producción) y debe mantenerse estrictamente separada del código fuente.

- **Variables de entorno**: Almacenan cadenas de conexión, credenciales de API, puertos y banderas de depuración.
- **Uso de archivos `.env`**: Archivos locales no subidos al control de versiones (`.gitignore`) para configuración en desarrollo.
- **Seguridad**: Evita quemar (*hardcodear*) secretos dentro de los repositorios de código.

Práctica: Inicialización de Proyectos

Entorno Node.js (ES Modules)

```
# Crear directorio y entrar
mkdir backend-app && cd backend-app

# Inicializar proyecto
npm init -y

# Configurar tipo módulo (package.json)
npm pkg set type="module"

# Instalar dependencias
npm install express dotenv
```

Entorno Python (FastAPI)

```
# Crear directorio y entrar
mkdir backend-app && cd backend-app

# Crear y activar entorno virtual
python -m venv venv
source venv/bin/activate

# Instalar dependencias
pip install fastapi uvicorn \
            pydantic-settings
```

Práctica: Estructura de Capas e IoC en Node.js

```
// repository/userRepo.js
export class UserRepository {
  async findById(id) { return db.query('SELECT * FROM users WHERE id = $1', [id]); }
}

// service/userService.js
export class UserService {
  constructor(userRepo) { this.userRepo = userRepo; } // DI por constructor
  async getUser(id) {
    const user = await this.userRepo.findById(id);
    if (!user) throw new Error("Usuario no encontrado");
    return user;
  }
}
```

Práctica: Controlador y Rutas en Node.js

```
// controller/userController.js
export class UserController {
  constructor(userService) { this.userService = userService; }
  async getProfile(req, res) {
    try {
      const user = await this.userService.getUser(req.params.id);
      res.json(user);
    } catch (err) { res.status(404).json({ error: err.message }); }
  }
}

// app.js - Inyección manual de dependencias
const repo = new UserRepository();
const service = new UserService(repo);
const controller = new UserController(service);
app.get('/users/:id', (req, res) => controller.getProfile(req, res));
```

Práctica: Inyección de Dependencias en Python (FastAPI)

```
from fastapi import FastAPI, Depends

class UserRepository:
    def get_by_id(self, user_id: int):
        return {"id": user_id, "nombre": "Carlos"}

# Proveedor de dependencia
def get_user_repo():
    return UserRepository()

app = FastAPI()

@app.get("/users/{user_id}")
def get_user(user_id: int, repo: UserRepository = Depends(get_user_repo)):
    user = repo.get_by_id(user_id)
    return user
```

Práctica: Manejo de Variables de Entorno (Node.js y Python)

Node.js (**dotenv**)

```
import dotenv from 'dotenv';
dotenv.config();

const config = {
  port: process.env.PORT || 3000,
  dbUrl: process.env.DATABASE_URL
};
```

Python (**pydantic-settings**)

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    port: int = 3000
    database_url: str

    class Config:
        env_file = ".env"

settings = Settings()
```

Estructura y gestión del código fuente

Estructura estándar de un proyecto back-end

Organización por capas o módulos

Una estructura de archivos bien definida facilita la escalabilidad, la mantenibilidad y la incorporación de nuevos desarrolladores al equipo.

- **`src/controllers/`**: Gestión de peticiones y respuestas HTTP.
- **`src/services/`**: Lógica de negocio pura e independiente del transporte.
- **`src/repositories/`** o **`models/`**: Acceso a datos y esquemas de persistencia.
- **`src/routes/`**: Definición de endpoints y asociación con middlewares/controladores.
- **`src/config/`**: Carga y validación de variables de entorno.
- **`src/middlewares/`**: Interceptores para autenticación, logs y validación.

Flujos de trabajo con control de versiones (Git)

Estrategias de ramificación (Branching Models)

Definen las reglas para la colaboración concurrente y la entrega continua de software.

- **Git Flow**: Adecuado para ciclos de publicación tradicionales. Maneja ramas permanentes (`main`, `develop`) y temporales (`feature/`, `release/`, `hotfix/`).
- **Trunk-Based Development**: Ideal para integración continua (CI/CD). Los desarrolladores integran pequeños cambios frecuentemente en la rama principal.
- **Convenios de Commits**: Uso de estándares como *Conventional Commits* (`feat:`, `fix:`, `refactor:`) para automatizar versiones y registros de cambios.

Gestión de secretos y configuración sensible

Seguridad en la gestión del código

Los datos sensibles nunca deben residir en el sistema de control de versiones.

- **Regla de oro**: El archivo `.env` debe agregarse inmediatamente a `.gitignore`.
- **Plantillas de configuración**: Se incluye un archivo `.env.example` con las claves necesarias pero sin valores reales.
- **Gestores de secretos en producción**: Uso de servicios dedicados (AWS Secrets Manager, HashiCorp Vault, Kubernetes Secrets) o variables de entorno inyectadas por la plataforma de despliegue.
- **Herramientas de escaneo**: Uso de `git-secrets` o `trufflehog` en pipelines para prevenir la subida accidental de llaves API o tokens.

Práctica: Configuración del repositorio y entorno

Inicialización de Git

```
# Inicializar repositorio local
git init -b main

# Crear plantilla de .gitignore
cat <<EOT > .gitignore
node_modules/
__pycache__/
venv/
.env
*.log
EOT

# Crear plantilla de entorno
touch .env .env.example
```

Instalación de paquetes

```
\tiny
# En Node.js (Estructura modular)
npm install express dotenv
npm install -D nodemon

# En Python (FastAPI + Pydantic)
pip install fastapi uvicorn \
            python-dotenv pydantic
pip freeze > requirements.txt
```

Práctica: Estructura de directorio del proyecto

```
mi-proyecto-backend/
├── .env
├── .env.example
├── .gitignore
├── package.json (o requirements.txt)
└── src/
    ├── config/
    │   └── env.js
    ├── controllers/
    │   └── user.controller.js
    ├── services/
    │   └── user.service.js
    ├── routes/
    │   └── user.routes.js
    └── app.js
```

Práctica: Gestión de configuración sensible en Node.js

```
// src/config/env.js
import dotenv from 'dotenv';
dotenv.config();

if (!process.env.DATABASE_URL) {
  throw new Error("ERROR CRÍTICO: FALTA LA VARIABLE DATABASE_URL");
}

export const config = {
  port: process.env.PORT || 3000,
  dbUrl: process.env.DATABASE_URL,
  apiSecret: process.env.API_SECRET
};
```

Práctica: Gestión de configuración sensible en Python

```
# src/config/env.py
import os
from dotenv import load_dotenv

load_dotenv()

class Config:
    PORT: int = int(os.getenv("PORT", 8000))
    DATABASE_URL: str = os.getenv("DATABASE_URL")

    if not DATABASE_URL:
        raise ValueError("ERROR CRÍTICO: FALTA LA VARIABLE DATABASE_URL")

config = Config()
```

Práctica: Punto de entrada estructurado (Node.js)

```
// src/app.js
import express from 'express';
import { config } from './config/env.js';

const app = express();
app.use(express.json());

app.get('/health', (req, res) => {
  res.json({ status: "OK", port: config.port });
});

app.listen(config.port, () => {
  console.log(`Servidor ejecutándose en el puerto ${config.port}`);
});
```

Práctica: Punto de entrada estructurado (Python)

```
# src/main.py
from fastapi import FastAPI
from config.env import config

app = FastAPI()

@app.get("/health")
def health_check():
    return {"status": "OK", "port": config.PORT}

# Ejecutar con: uvicorn main:app --port 8000 --reload
```